An Explainable Distributed Deep Learning Framework for Financial Risk Management

By:
23I-2649 Sabeel Nadeem
23I-2591 Lubabah Moten
23I-2626 Ibrahim Hussain

1. Introduction

Predicting financial asset prices is one of the toughest problems in machine learning. Markets are noisy, unpredictable, and constantly changing. They're also influenced by countless factors that are nearly impossible to measure. Classical statistical models like ARIMA and VAR work reasonably well when markets are calm and stable, such as the foreign exchange market, but they fall apart when multiple assets are affected by mutual influence and mass human behaviour. LSTM networks were a big step forward. They finally gave us a way to capture temporal patterns that older models couldn't see. But LSTMs have a major blind spot: they treat each asset in isolation. Anyone who follows markets knows that when Bitcoin drops, Ethereum usually follows. When EUR/USD moves, GBP/USD tends to move with it. LSTMs don't model these spatial relationships at all. Graph neural networks (GNNs), on the other hand, are built specifically to handle exactly this kind of problem.

The paper we're reproducing is called "Financial Asset Price Prediction with Graph Neural Network-Based Temporal Deep Learning Models" by Uygun and Sefer (2025). The authors apply three GNN-based models: MTGNN, StemGNN, and FourierGNN to predict prices in Forex and Cryptocurrency markets. They compare these against LSTM, ARIMA, and VAR. Their main argument is simple: by treating assets as nodes in a graph and letting the model learn how they influence each other from the data, GNNs can capture dependencies that traditional approaches completely miss.

Our reproduction focuses mainly on FourierGNN, though we also ran experiments with MTGNN. What makes FourierGNN interesting is that it takes a completely different architectural approach from the other two. Instead of using separate modules to handle spatial and temporal modeling, it treats the entire input every value for every asset at every time step as one giant graph. Then it processes everything in the frequency domain using the Fourier Graph Operators. This sounds complicated, but it actually makes the computation feasible even with a huge number of nodes.

We ran experiments on both datasets: Forex with 10 currency pairs and Cryptocurrency with 20 assets. We followed the same rolling window backtesting protocol described in the paper, and our hyperparameters match exactly what the paper reports in Table 2. This report walks through our methodology, presents our results, and discusses the practical challenges we ran into while trying to reproduce the original work.

2. Paper Overview

- ## 2.1 Problem Statement

  The paper investigates whether graph neural networks can outperform traditional time-series models for financial price prediction across two markets: Forex (foreign exchange) and Cryptocurrency. These markets are deliberately contrasted because Forex is relatively stable and liquid, while Crypto is highly volatile. The hypothesis is that GNNs should perform especially well in volatile markets, where inter-asset dependencies provide additional predictive signal.
  The prediction task is return ratio forecasting. Rather than predicting raw prices, models predict how much the price changes relative to a lookback window, making the series more stationary. The horizon is 5 days for Forex and 7 days for Cryptocurrency.

- ## 2.2 Datasets Used

  Cryptocurrency Dataset:

  - 20 assets selected from the top 50 by market capitalisation
  - Daily closing prices from October 2016 to October 2022
  - Assets include BTC-USD, ETH-USD, XRP-USD, ADA-USD, DOGE-USD, and 15 others
  - About 15% of values were backfilled due to some assets having less historical data
  - Total of approximately 2,190 daily observations per asset

  Forex Dataset:

  - 10 major currency pairs
  - Daily closing prices from May 2006 to October 2023
  - Pairs include EUR-USD, GBP-USD, JPY-USD, AUD-USD, and 6 others
  - No missing values
  - Total of approximately 5,118 daily observations per asset

  Both datasets were downloaded using the yfinance library and preprocessed by the authors. In our reproduction, we used the preprocessed CSV files directly — daily_20_2190_marked.csv for crypto and daily_10_5118_marked.csv for Forex. The _marked suffix indicates that the return ratio transformation and normalisation have already been applied.

- ## 2.3 Models Compared (MTGNN, StemGNN, FourierGNN, LSTM, ARIMA, VAR)

Model	Type	Characteristics
MTGNN	GNN	Learns adaptive graph structure via attention. Combines graph convolution with dilated temporal convolutions
StemGNN	GNN	Spectral domain processing uses the graph Fourier transform for spatial and DFT for temporal dependencies
FourierGNN	GNN	Hypervariate graph; process all values jointly in the frequency domain using Fourier graph operators.

- ## 2.4 Evaluation Methodology

  For price forecasting, the measures MAPE, MAE, and RMSE are considered, with low scores representing good performance. In addition, the A20 score is employed to assess the fraction of the forecasts falling into 20% of the true prices, with high scores representing good performance. The A20 score is, however, discarded in the StemGNN model.
  The accuracy and other metrics in terms of classifying stock performance directions are calculated in a three-class classification framework (Buy, Hold, and Sell). Any price increase exceeding +1% is categorised as Buy, any decline below -1% is classified as Sell, while values between -1% and +1% are referred to as Hold. Performance of portfolios based on predictions generated by the model is measured by annualised return, annualised volatility, Sharpe ratio, and maximum drawdown.
  Backtesting is done through rolling windows with a length of 104 weeks. At each time step, the algorithm learns from all data preceding the last 104 weeks; 25% of the dataset is used for validation, after which predictions for the next week are generated. The results are reported as an average value across 10 experiments.

3. Implementation Details

- ## 3.1 Code Repository and Framework

  We used the official repository from the paper, temporal_gnn. The repository contains implementations of all three GNN models along with data loading and training scripts. Key files used were:

- main.py (implementation file for FourierGNN.py, MTGNN implementation, StemGNN)

The main tools used are:

- Python 3.12.7 (via pyenv as the new Ubuntu version does not come withitrh this)
- PyTorch 2.2.2 with CUDA 12.1 support
- TensorFlow 2.16.1
- Hyperopt 0.2.7 (deprecated library, so we had to implement some fixes)

3.2 Model FourierGNN

FourierGNN presents the idea of the hypervariate graph. Given N time series and time horizon T, every value in the input is one graph node, meaning there will be N×T nodes altogether — completely interconnected in a complete graph. This leads to 150 nodes in the case of Forex experiments (N=10, T=15); in the case of Crypto (N=20, T=7), there will be 140 nodes in the graphs. FourierGNN maps node embeddings to a d-dimensional space, then applies the Discrete Fourier Transform to transition to the frequency domain, performs computations by passing the data through stacked layers of the Fourier Graph Operator (FGO) with nonlinear activation functions, applies the Inverse Discrete Fourier Transform to transition back to the time domain, and produces predictions through a feed-forward neural network with fully-connected layers. Frequency-based operations eliminate costly computations of dense graph convolutions by replacing them with simple element-wise multiplication operations.

3.3 Model MTGNN

MTGNN works by learning its own graph structure from scratch. It figures out which assets are related to each other by comparing how similar their node embeddings are. Once the graph is built, the spatial convolution layers capture how different assets influence one another, while dilated temporal convolutions track patterns across different time scales. The model keeps switching back and forth between these two types of convolution. This lets it do two things at once: understand how assets move together, and understand how each asset behaves over time. For the Crypto dataset, we ran a hyperparameter search and found the best settings were: conv_channels = 16, epoch = 30, layers = 3, learning rate = 8.63e-4, num_of_weeks_in_window = 5, and weight_decay = 1.08e-5. The final model has about 90,000 parameters and a receptive field of 187 time steps.

- ## 3.3 Model StemGNN

  StemGNN is completely based on the spectral domain approach. The StemGNN model uses the Graph Fourier Transform to extract spatial dependency (dependency between different series) and the Discrete Fourier Transform to extract temporal dependency. Since StemGNN works in the frequency domain on both dimensions (spatial and temporal), it becomes easier to extract short-term information from long-term information. For StemGNN analysis, I used the implementation from the repository itself on both datasets.

- ## 3.3 Data Pipeline

  The data pipeline begins with raw price collection using the Yahoo Finance library (yfinance), which fetches daily closing prices for selected cryptocurrency and foreign exchange (Forex) assets over a multi-year period. To address API instability, the library was patched to bypass authentication issues and use a direct JSON endpoint, ensuring reliable data retrieval. The raw price series are non-stationary, so they are transformed into return ratios—calculated as the price at time t divided by the price at time t−h, where h is the forecasting horizon (7 days for crypto, 5 days for Forex). These return ratios are then normalised by the maximum absolute value across the series to stabilise variance and improve model convergence.
  The processed data is structured as a multivariate time series, with each asset as a separate channel. A sliding window approach is used to create input sequences of length T (e.g., 15 days) to predict the next H steps (e.g., 7 days). The data is split chronologically into training (70%), validation (20%), and test (10%) sets. For backtesting, a rolling window scheme simulates weekly retraining over 104 weeks, where the model is re-fit on expanding historical data and evaluated on the following week. Final outputs include prediction CSV files and evaluation metrics (MAPE, MAE, RMSE, A20), which are later aggregated for portfolio analysis. This pipeline ensures that all models are compared under identical data conditions.

4. Experimental Setup

- ## 4.1 Hardware Configuration
  - OS: Ubuntu 22/24.04 (WSL & Native Installation)
  - RAM: 4-12 GB
  - External/Cloud compute used was NVIDIA Tesla T4 GPU with CUDA 12.1 support, 16 GB VRAM, 2560 CUDA Cores
  - Storage: Offline & git-lfs
  - Version Control: git & git-lfs

- ## 4.2 Dataset Preprocessing

  Raw price data from Yahoo Finance exhibited non-stationarity, which we addressed by transforming prices into return ratios: $RR_{\square} = P_{\square} / P_{\square-\square}$ with horizons of 5 days for Forex (weekday-only trading) and 7 days for cryptocurrencies (continuous trading). Return ratios were chosen over log-returns for three key advantages: intuitive interpretation (e.g., 1.05 represents a clear 5% gain), preservation of multiplicative structure for portfolio compounding, and robustness to zero or negative prices where log-returns fail. To normalise across assets, each series was divided by its maximum absolute value, bounding values within [-1, 1] to improve neural network convergence. For cryptocurrency assets with limited historical data (e.g., FIL-USD, MKR-USD), we backfilled missing values—approximately 15% of the dataset—rather than discarding them, as the latter produced marginally worse performance in validation tests. The Forex dataset contained no missing entries. Finally, a sliding window approach was applied: each input window of sequence length *s* (hyperparameter, typically 7–15 days) was used to forecast the target price at horizon *h* (5 or 7 days ahead). This produced multivariate input-output pairs for training, validation, and testing across all assets.

- ## 4.3 Hyperparameters

To get the best possible performance from each model, we ran hyperparameter optimisation using Hyperopt. For the deep learning models, we picked the hyperparameters that gave us the lowest validation loss. The parameter names are kept exactly as they appear in the code implementations.

For FourierGNN, the Forex setup used a sequence length of 15, embed size of 32, hidden size of 64, learning rate of 9.59×10^{-4}, exponential decay step of 5, and decay rate of 0.8. For Crypto, the settings were sequence length 7, embed size 8, hidden size 128, learning rate 1.15×10^{-4}, exponential decay step 8, and decay rate 0.9.

MTGNN used a sequence length of 10 for Forex and 35 for Crypto, with 3 layers and 4 conv channels for both datasets. The learning rates were 1.42×10^{-4} for Forex and 6.35×10^{-4} for Crypto, with weight decay set to 3.49×10^{-5} and 2.03×10^{-5}, respectively.

For StemGNN, Forex used sequence length 15, multi-layer 13, learning rate 0.01, exponential decay step 13, decay rate 0.5, and dropout rate 0.75. Crypto used sequence length 7, multi-layer 8, learning rate 0.001, exponential decay step 8, decay rate 0.5, and dropout rate 0.5.s

- ## 4.4 Training Configuration

All models shared the same training configuration. We used RMSprop with epsilon set to 1e-8, MSELoss with mean reduction, and an ExponentialLR scheduler for the learning rate. Early stopping kicked in if validation loss didn't improve for 20 epochs. For data loading, we shuffled the training and validation sets but left the test set untouched. Hyperparameter search was done using Hyperopt with the TPE algorithm. Rolling window backtesting ran from week 2 all the way to week 104. Hyperparameter search ran at week 2, then repeated every 10 weeks after that, specifically at weeks 10, 20, 30, 40, 50, 60, 70, 80, 90, and 100. The data was split with 70% for training, 20% for validation, and the remaining 10% for testing.

5. Reproduced Results

- ## 5.1 FourierGNN on optimising

Metrix	Week 104 result	Paper Reference

MAPE(%)	1.37%	1.5%
MAE	0.01361	0.015
RMSE	0.02273	0.03
A20 Index	1.00	0.95

The Forex results are remarkably strong. An A20 Index of 1.000 at week 104 means every single prediction in the test set fell within 20% of the actual value. The MAPE of 1.37% is close to and slightly better than the paper's reported 1.5%, confirming that FourierGNN reproduces very well on the more stable Forex market. This is consistent with the paper's finding that FourierGNN excels on Forex due to the relatively smooth, mean-reverting nature of currency returns compared to crypto. The early stopping occurred at epoch 87, indicating the model converged well within the 100-epoch budget

- ## 5.2 FourierGNN on Cryptocurrency

The A20 Index of approximately 0.87 indicates that 87% of predictions fall within 20% of the actual value, exceeding the paper's reported 0.75. The MAPE values (11.3-11.4%) are higher than the paper's ~5%, which is expected given the reduced epochs (50 vs 100) and fewer hyperparameter search trials (10 vs 100) in our FourierGNN runs. The observation on training is the improvement from epoch 1 through convergence. At the start of training at week 104, the validation MAPE was 127%, reducing steadily to 123% before early stopping was triggered at epoch 48. This shows the model is learning despite the noisy crypto returns.

- ## 5.3 MTGNN on Cryptocurrency

The MTGNN hyperparameter search over 100 trials on the Cryptocurrency dataset ran for approximately 12 hours and 51 minutes total. The search converged to a best validation loss of 0.0341, with optimal hyperparameters: conv_channels=16, layers=3, learning rate=8.63e-4, num_of_weeks_in_window=5, weight_decay=1.08e-5, epoch=30. The final trial (trial 100) achieved its best checkpoint: test MAPE = 13.30%, test MAE = 0.1363, test RMSE = 0.1799, test A20 = 0.7772. Across the 30-epoch training run, validation MAPE decreased from ~107% at epoch 1 to ~1.35% at epoch 30, showing the model learned to exploit the return ratio normalisation effectively.

- ## 5.4 StemGNN on Forex

StemGNN was also run on the Forex dataset using the same hyperparameter configuration. The Forex model has approximately 10,518,642 trainable parameters,

slightly fewer than the Crypto model due to the smaller number of assets (10 currency pairs vs. 20 crypto assets). Three weeks of the rolling window were completed at the time of writing (weeks 2, 3, and 4), with the run ongoing. All three weeks completed the full 100 epochs with no early stopping, taking approximately 102.7, 84.6, and 60.0 minutes, respectively. Test MAPE for the three weeks was 1.27%, 1.27%, and 0.97%, with MAE values of 0.01263, 0.01272, and 0.00972, and RMSE values of 0.02098, 0.02293, and 0.01473.

- ## 5.5 StemGNN on CryptoCurrency

  StemGNN was run on the Cryptocurrency dataset using the repository's StemGNN implementation, following the same rolling window backtesting protocol. The model was configured with 100 epochs, a window size of 15, a horizon of 7, a learning rate of 0.01, a multi-layer of 13, an exponential decay step of 13, a decay rate of 0.5, a dropout rate of 0.75, and a batch size of 32, matching the hyperparameters reported in the paper. The model has approximately 10,520,072 trainable parameters and was run on a T4 Nvidia GPU with 16 GB RAM via Google Colab. Training at week 104 completed all 100 epochs in approximately 54.7 minutes, with each epoch taking roughly 29–35 seconds. The week 104 test results are: MAPE 9.29%, MAE 0.0855, RMSE 0.1219.

- ## 5.6 Comparison Table

Metric	MTGNN	FourierGNN	StemGNN
MAPE	13.30%	11.43%	9.29%
MAE	0.1383	0.1009	0.0855
RMSE	0.1799	0.1373	0.1219
A20 Index	0.7772	0.8668	-

6. Reproducibility Discussion

- ## 6.1 Differences from Original Results

  Several systematic differences between our setup and the original paper likely affected our results. First, we reduced training epochs from 100 to 50 for FourierGNN. The original paper had a multiplier of ×2 in their training loop, which we removed to keep training exactly at 50 epochs. Fewer epochs mean less time for the model to converge, especially for a complex architecture like FourierGNN.

Second, we had to cut hyperparameter search trials for FourierGNN from 100 down to just 10 due to computational constraints. With only 10 trials, we're probably finding reasonable configurations, but we're unlikely to find truly optimal ones consistently. MTGNN, on the other hand, got the full 100 trials, which explains why its results are better calibrated despite having higher absolute error.

Third, while our rolling window backtesting covers the full 104 weeks just like the paper does, our reductions in epochs and HPO trials mean each week's model is trained with significantly less compute than what the original authors used. Finally, there's a data split difference. The code we worked with uses train_ratio=0.7 and val_ratio=0.2, but the paper describes a 74%/24% split for the hyperparameter search phase. This means our models get slightly less training data for each window.

- ## 6.2 Implementation Challenges

1. Runtime was the biggest practical challenge we faced. Running MTGNN hyperparameter search with 100 trials for a single week on the Crypto dataset took about 12 hours and 51 minutes. Early on, we had a scare when one epoch seemed to stall for nearly 7 hours. Turns out the laptop went to sleep mid-run, which reset the GPU state and forced that epoch to run on CPU instead. Lesson learned: always disable system sleep before starting long training jobs.
2. We also ran into a pkg_resources error on our first run. Hyperopt's atpe module was trying to import from pkg_resources, but that module was removed from newer versions of setuptools. Reinstalling setuptools with --force-reinstall fixed the error, as well as replacing the ‘import pkg_resources’ with ‘import importlib. resources as pkg_resources’
3. Package installation was another headache. Large packages like nvidia-cusparse-cu12 (196 MB), nvidia-cudnn-cu12 (732 MB), torch (755 MB), and tensorflow (590 MB) kept timing out. Adding --timeout 1000 to the pip install command finally did the trick.
4. Finally, we kept seeing division by zero warnings from permetrics/regression.py throughout training. This happens when actual return ratio values are zero or close to zero. It doesn't stop training, but it does mean some MAPE values during validation are unreliable. The test evaluation seems to handle these cases more cleanly.

- ## 6.3 Environment and Dataset Differences

We ran our experiments on Python 3.12.7 and PyTorch 2.2.2. The paper doesn't specify which versions were used, which is worth noting because floating-point operations can produce slightly different results across PyTorch versions due to internal optimisation changes, even when using the same random seed. Speaking of seeds, neither the paper nor the provided code sets a global random seed. Instead, they handle stochasticity by averaging results over 10 independent runs. Rather than re-downloading and processing the raw data ourselves, we used the pre-processed CSV files. We trusted that the authors' preprocessing matched what was described in the paper. The _marked suffix in the filenames tells us that the return ratio transformation and normalisation have already been applied. One more detail about our setup: we ran everything on Linux due to the effective scheduling of the OS and also due to the reason that it developement in Linux is much more preferable and widely adopted than any other OS. Running code on WSL and storing its results added disk I/O overhead compared to running on native Linux. It didn't affect the correctness of our results, but it definitely made the runs take longer.

7. Conclusion

We successfully reproduced the experiments for FourierGNN, MTGNN, and StemGNN from Uygun & Sefer (2025) on both Forex and Cryptocurrency datasets.

For the Forex dataset, the results are very close to those reported in the paper. FourierGNN achieved a MAPE of 1.37% and an A20 score of 1.000 at week 104, compared to the paper’s 1.5% MAPE and 0.95 A20. This close alignment indicates that the model’s frequency-domain approach works particularly well for the relatively stable and predictable nature of Forex markets.

On the Cryptocurrency dataset, the MAPE is higher than reported in the paper (around 11.4% compared to approximately 5.0%), while the A20 score is higher (around 0.87 compared to 0.75). The increase in MAPE can be explained by the reduced number of training epochs and fewer hyperparameter optimisation trials. However, the stronger A20 score suggests that, despite less accurate magnitude predictions, the model still performs well in keeping predictions within an acceptable error range. This may be due to the reduced training acting as a form of regularisation, limiting overfitting.

MTGNN achieved an MAPE of 13.30% and an A20 score of 0.7772 on the Cryptocurrency dataset, compared to FourierGNN’s 11.43% MAPE and 0.8668 A20. These results confirm that FourierGNN performs better than MTGNN in this setup, which is consistent with the findings reported in the original paper.

A key takeaway from this reproduction is the high computational cost of the full experimental setup. Conducting 100 hyperparameter search trials every 10th week over a 104-week backtesting period is not practical on standard personal hardware within typical project constraints. As a result, any reproduction effort needs to balance the number of trials, training epochs, and backtesting duration with the available computational resources.

From a modelling perspective, FourierGNN's design—processing all assets and time steps together in the frequency domain—appears to provide a meaningful advantage over approaches that handle spatial and temporal dependencies separately. Overall, the results support the paper's main argument that graph neural network-based models can outperform traditional statistical methods and standard LSTM-based approaches for financial time-series prediction, especially in more stable market conditions.

8. Code Repository:

[fin-glassbox]

9. References

- Uygun, Y., & Sefer, E. (2025). Financial asset price prediction with graph neural network-based temporal deep learning models. Neural Computing and Applications, 37, 25445-25471.
- Yi, K., Zhang, Q., Fan, W., He, H., Hu, L., Wang, P., An, N., Cao, L., & Niu, Z. (2023). FourierGNN: Rethinking multivariate time series forecasting from a pure graph perspective. Advances in Neural Information Processing Systems, 36, 69638-69660.
- Wu, Z., Pan, S., Long, G., Jiang, J., Chang, X., & Zhang, C. (2020). Connecting the dots: Multivariate time series forecasting with graph neural networks. Proceedings of the 26th ACM SIGKDD, 753-763.
- Cao, D., Wang, Y., Duan, J., Zhang, C., Zhu, X., Huang, C., Tong, Y., Xu, B., Bai, J., & Tong, J. (2020). Spectral temporal graph neural network for multivariate time-series forecasting. Advances in Neural Information Processing Systems, 33, 17766-17778.
- Bergstra, J., Yamins, D., & Cox, D. D. (2013). Hyperopt: A Python library for optimising the hyperparameters of machine learning algorithms. Proceedings of the 12th Python in Science Conference, 13, 20.
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. European Journal of Operational Research, 270(2), 654-669.
- Aroussi, R. (2024). yfinance: Yahoo! Finance market data downloader. https://github.com/ranaroussi/yfinance